TOGGLEMASTER

Relatório final de implantação — Tech Challenge Fase 2

FIAP • Pós-Tech DevOps | Revisão documental: 02/11/2026

Ambiente acadêmico AWS Academy — LAB 2. Conta 533267349027, região us-east-1. Este relatório descreve a implantação, as decisões técnicas e os resultados registrados nas evidências do projeto.

Equipe

Integrante	RM	Discord
Afonso Victoriano Franco	RM373563	afonso3254
Cesar Augusto Ferreira Alves	[PREENCHER]	cesarf__
Jeferson Lima Papalino	RM376680	jefersonpapalino
Jonatas Ariel Nunes	RM376145	eusoujoe
Willian Felix Corrêa	RM372688	will1an2522

Links de entrega

Repositório do projeto: https://github.com/jefersonpapalinosp-sys/Tech-challenge-grupo-2

Código da Fase 2: https://github.com/jefersonpapalinosp-sys/Tech-challenge-grupo-2/tree/main/fase2

Vídeo de demonstração: [INSERIR LINK DO VÍDEO]

Duração do vídeo exportado: 18:19

O vídeo editado foi conferido em 02/11/2026, por quadros de cada trecho e transcrição do áudio. O link será acrescentado após a publicação.

> *Os RMs de quatro integrantes e os usernames do Discord dos cinco foram informados pela equipe. Faltam o RM de Cesar Augusto Ferreira Alves e o link do vídeo; preencher antes do envio à FIAP.*

Referência e escopo

Enunciado: POSTECH - Tech Challenge - Fase 2, 11 páginas, disponível em docs. A organização segue os cinco requisitos técnicos e os entregáveis das páginas 9–10.

Fontes: Dockerfiles, Docker Compose, manifestos k8s-aws, coletas de 30 e 31/10/2026 e gravação do ensaio de analytics de 02/11/2026. Estados e resultados devem ser interpretados nas datas de cada coleta.

1. Visão geral

Arquitetura e responsabilidades

O ToggleMaster evoluiu de um monólito para cinco microsserviços com implantação e escala independentes. O caminho de avaliação responde à aplicação cliente, enquanto o processamento analítico ocorre de forma assíncrona.

Serviço	Tecnologia / porta interna	Responsabilidade
auth-service	Go / 8001	Criar e validar chaves de API; persistir no PostgreSQL de autenticação.
flag-service	Python / 8002	Manter definições e estado das feature flags.
targeting-service	Python / 8003	Manter regras de segmentação e percentuais de liberação.
evaluation-service	Go / 8004	Consultar flag/regra, utilizar cache Redis, decidir e publicar eventos.
analytics-service	Python / 8005	Consumir SQS e persistir eventos no DynamoDB.

Fluxo principal

```
Cliente / Postman
  -> Classic Load Balancer -> ingress-nginx -> /evaluate
  -> evaluation-service -> flag-service + targeting-service
                        -> Redis (cache)
                        -> SQS tgmqueue
                           -> analytics-service
                              -> DynamoDB ToggleMasterAnalytics
```

Auth valida as chaves utilizadas nas operações protegidas de flags e regras. O cache do evaluation armazena os dados combinados de flag e regra por 30 segundos. A publicação do evento é disparada em uma goroutine; a resposta HTTP não aguarda a persistência analítica.

Diferença entre os três armazenamentos — enunciado, p. 9

Armazenamento	Propósito e uso no projeto
RDS PostgreSQL	Fonte persistente dos dados relacionais de autenticação, flags e regras. Três instâncias separam os domínios; os serviços responsáveis consultam e atualizam essas informações.
ElastiCache Redis	Cache em memória dos dados combinados de flag e regra, com TTL de 30 s. Em cache miss, o evaluation consulta os serviços e recompõe o cache.
DynamoDB	Armazenamento NoSQL dos eventos analíticos consumidos da SQS. Cada registro usa event_id como chave e inclui flag_name, user_id, resultado e timestamp.

RDS mantém o estado de negócio; Redis reduz a latência e as consultas repetidas; DynamoDB preserva os eventos para análise. A separação permite consultar flags e processar analytics de forma independente. Alterações em flags ou regras podem aguardar a expiração do cache para aparecer na avaliação.

HTTP 200 e result=true confirmam a avaliação. Para confirmar o fluxo completo, é necessário correlacionar o evento persistido no DynamoDB com o identificador do teste.

2. Análise e conteinerização

Requisito 1 — Dockerfiles e ambiente local

Os cinco serviços têm Dockerfiles multi-stage. Nos serviços Go, a compilação ocorre no builder e o binário segue para uma imagem Alpine de execução. Nos serviços Python, as dependências são preparadas no builder e copiadas para a imagem final. As aplicações executam como usuário não root.

O docker-compose.yml da raiz reúne nove contêineres. As dependências possuem health checks; as aplicações aguardam serviços essenciais saudáveis por depends_on.

Contêiner / grupo	Quantidade	Função / acesso no computador
auth-service	1	http://localhost:18001
flag-service	1	http://localhost:18002
targeting-service	1	http://localhost:18003
evaluation-service	1	http://localhost:18004
analytics-service	1	http://localhost:18005/health
auth-db + apps-db	2	PostgreSQL: autenticação e bancos flags/targeting.
redis	1	Cache Redis local.
aws-local	1	LocalStack: SQS e DynamoDB; porta 4566.
Total	9	Cinco aplicações e quatro contêineres de infraestrutura.

Validação local registrada em 31/10

A verificação executada nesta sessão encontrou os nove contêineres saudáveis. scripts/test-local.ps1 confirmou os cinco endpoints /health, criação e validação de chave, criação de flag e regra PERCENTAGE=100, avaliação true e aumento de 5 para 6 itens no DynamoDB local.

O critério do script local para persistência é aumento de contagem total; para evidências com tráfego concorrente, deve-se complementar a contagem com a correlação da flag do próprio teste.

Adaptação ao enunciado: foi usado LocalStack para emular DynamoDB e SQS em um contêiner. Isso mantém nove contêineres e permite o fluxo assíncrono local, mas não é a imagem DynamoDB Local citada literalmente. A adaptação é declarada; sua aceitação pelo avaliador não foi presumida.

3. Infraestrutura AWS

Requisito 2 — inventário do LAB 2

A coleta de infraestrutura de 30/10/2026, entre 22:04:41 e 22:05:20 BRT, identificou os recursos abaixo. As evidências brutas estão na pasta evidencias que acompanha os documentos de entrega.

Recurso	Identificação / configuração registrada
Conta e região	533267349027 • us-east-1
VPC	vpc-09d66ad83422a42ee
EKS	ekstogglemaster • ACTIVE • Kubernetes 1.37
Managed Node Group	tgmnodegroup • ACTIVE • t3.medium • mínimo 1 / desejado 2 / máximo 4
Nós na coleta	Dois nós Ready. A configuração 1/2/4 não é prova de Cluster Autoscaler instalado.
RDS Auth	db-authservice • disponível • PostgreSQL • privado
RDS Flag	dbflags • disponível • PostgreSQL • privado
RDS Targeting	dbtargeting • disponível • PostgreSQL • privado
ElastiCache	redisevaluation • disponível • cluster mode • 1 shard • TLS/AUTH e criptografia em repouso
SQS	tgmqueue • Standard • criptografia gerenciada pela AWS
DynamoDB	ToggleMasterAnalytics • ACTIVE • PAY_PER_REQUEST • chave de partição event_id

Imagens privadas no ECR

Prefixo: 533267349027.dkr.ecr.us-east-1.amazonaws.com/

Aplicação	Repositório e tag
auth-service	namespace/auth-service:v1
flag-service	namespace/lag-service:v1
targeting-service	namespace/targeting-service:v1
evaluation-service	namespace/evaluation-service:v3
analytics-service	namespace/analytics-service:v1

namespace/lag-service é o nome real do repositório utilizado pelo flag-service. A tag v3 do evaluation incorpora o cliente go-redis/v9 utilizado com Redis em cluster mode. A coleta do ECR registrou scanOnPush=false, mantido como ponto de melhoria.

4. Rede e configuração

Conectividade, segredos e identidade de execução

RDS e Redis são acessados por endpoints privados. Nos três grupos dos RDS há entrada TCP 5432 para o Security Group do EKS sg-079be737f17db5c6b e uma entrada administrativa adicional 152.245.x.x/32, mascarada nesta versão. Portanto, a origem autorizada não é exclusivamente o EKS.

O Redis permite TCP 6379 a partir do SG do EKS; a coleta também registra uma regra autorreferente do SG default. A URL de conexão utiliza TLS e token, sem publicar seu valor no relatório.

Segredo no AWS Secrets Manager	Destino Kubernetes
togglemaster/auth	auth-service / auth-secret
togglemaster/flag	flag-service / flag-secret
togglemaster/targeting	targeting-service / targeting-secret
togglemaster/evaluation	evaluation-service / evaluation-secret

O ClusterSecretStore aws-secrets-manager conecta o External Secrets Operator ao Secrets Manager. A coleta registrou store válido e os quatro ExternalSecrets com SecretSynced. ConfigMaps contêm URLs internas, região, fila e nome da tabela; os valores sensíveis são injetados por Secrets.

Decisão de identidade no AWS Academy

A configuração documentada utiliza LabRole via credenciais do nó e IMDSv2, com hop limit 2 para viabilizar o acesso dos workloads no laboratório. Isso não isola permissões IAM por aplicação. A leitura estática dos manifestos não comprova o ARN atual da role nem a persistência dessa configuração em nós recriados; a verificação deve acompanhar o ciclo de vida do LAB.

Isso foi observado em 31/10 e 01/11/2026: instâncias recriadas pelo node group subiram com hop limit 1 e não entraram no cluster. O log de inicialização registrou a negação de ec2:DescribeInstances por uma política da organização, comportamento compatível com o encerramento da sessão do LAB. Com o LAB ativo, o hop limit foi ajustado para 2, mantendo IMDSv2 obrigatório, e as instâncias foram reiniciadas; os nós voltaram a Ready.

Para um ambiente de produção, a evolução indicada é identidade por workload com Pod Identity ou IRSA, permissões mínimas e entrada HTTPS. Esses itens são recomendações, não funcionalidades declaradas como implantadas.

Base64 em Kubernetes Secret é codificação reversível, não criptografia. O pacote contém nomes e metadados de recursos, sem senhas, tokens ou valores de Secrets.

5. Cluster e implantação

Requisitos 3 e 4 — componentes e manifestos

Componente	Papel e evidência
Metrics Server	Fornece métricas de recursos. Os HPAs coletados possuem valores de CPU, demonstrando disponibilidade de métricas naquele momento.
External Secrets Operator	Sincroniza quatro segredos; store Valid e ExternalSecrets SecretSynced.
ingress-nginx	Service my-release-ingress-nginx-controller, namespace ingress-nginx; recebe o tráfego do Load Balancer.

Organização Kubernetes

Os namespaces finais são auth-service, flag-service, targeting-service, evaluation-service e analytics-service. Os Kustomizations aplicam esses nomes aos arquivos base; por isso, executar um arquivo base isoladamente com -f pode produzir um namespace diferente do manifesto final.

Cada aplicação possui Deployment, Service ClusterIP e ConfigMap. As configurações sensíveis são sincronizadas por ExternalSecrets. Os Deployments definem requests/limits, usuário não root, bloqueio de elevação de privilégios, capabilities removidas e sistema de arquivos raiz somente leitura.

Startup, readiness e liveness controlam inicialização, recebimento de tráfego e recuperação. No analytics, readiness utiliza HTTP /health; startup e liveness usam TCP. Jobs SQL inicializam os três bancos; seu TTL permite que objetos concluídos sejam removidos posteriormente.

Rota externa	Destino
/auth	auth-service:8001, com reescrita do prefixo pelo Ingress
/flags	flag-service:8002
/rules	targeting-service:8003
/evaluate	evaluation-service:8004

O analytics é um worker interno e não possui rota pública no Ingress. A coleta Kubernetes registrou cinco Deployments disponíveis e quatro rotas Ingress.

O LAB 2 utiliza Classic Load Balancer. O enunciado menciona ALB/NLB; a divergência permanece declarada e não deve ser apresentada como aderência literal. A revisão documental não alterou o recurso.

6. Escalabilidade

Requisito 5 e entregável da página 9 — implementação e justificativa

HPA	Métrica / alvo	Réplicas / estabilização
evaluation-service-hpa	CPU média: 30% do request	Mínimo 1 • máximo 5 • scale-down 60 s
analytics-service-hpa	CPU média: 30% do request	Mínimo 1 • máximo 5 • scale-down 60 s

Como a escala do analytics foi implementada

k8s-aws/analytics/hpa.yaml define um HorizontalPodAutoscaler autoscaling/v2 que controla o Deployment analytics-service. O Kustomize aplica o namespace final analytics-service. O HPA consulta as métricas de CPU disponibilizadas pelo Metrics Server e ajusta a quantidade de réplicas entre 1 e 5.

O Deployment solicita requests.cpu=100m por pod. Com averageUtilization=30, o alvo corresponde a 30m de CPU por pod em média, equivalentes a 0,03 vCPU. O controlador compara a utilização observada com esse alvo para calcular a quantidade desejada de réplicas, respeitando seus limites e tolerâncias. A janela scaleDown.stabilizationWindowSeconds=60 reduz oscilações na redução de réplicas; ela não garante retorno ao mínimo exatamente após 60 segundos.

Os pods do analytics consomem a mesma fila tgmqueue, processam os eventos e gravam no DynamoDB. Mais mensagens podem aumentar o trabalho e a CPU, provocando escala. Porém, esperas de rede ou baixo custo de processamento podem manter a CPU abaixo do alvo mesmo com mensagens acumuladas. O número de mensagens na fila não é uma métrica deste HPA.

Por que HPA por CPU foi escolhido

A escolha segue o requisito mínimo da opção AWS Academy descrito na página 8 do enunciado. Ela aproveita o Metrics Server já instalado e a identidade disponível no laboratório, sem adicionar um operador e a configuração de acesso à SQS para obter métricas de fila. A CPU funciona como indicador indireto da carga do worker; essa limitação foi considerada na escolha.

KEDA é a alternativa opcional indicada no enunciado, recomendada para a opção de conta pessoal. Um ScaledObject com scaler SQS pode ajustar réplicas pela profundidade da fila e permitir escala a partir de zero conforme a configuração. Não há implantação de KEDA neste LAB; o HPA atual mantém ao menos uma réplica. A escolha por HPA não significa que KEDA seja tecnicamente impossível no Academy.

Implementação e resultado são verificações distintas. O aumento de réplicas do analytics foi comprovado no ensaio de 02/11/2026. A página seguinte apresenta somente os resultados efetivamente registrados.

6. Escalabilidade — resultados

Ensaios registrados e limites da demonstração

Ensaio mais recente de evaluation — 31/10/2026

Fonte: evidência 13, entre 00:59:22 e 01:02:31 BRT. Flag usada: smoke-aws-1793419004. Foram criados dois pods temporários de carga, removidos ao encerrar o ensaio.

Etapa	Registro observado
Antes	evaluation Deployment 1/1; CPU 1% / alvo 30%. Analytics em uma réplica.
Aumento inicial	CPU 59%; HPA passou a desejar duas réplicas.
Snapshot durante a carga	CPU 172%; Deployment 2/5. Cinco pods Running, dos quais dois Ready naquele instante.
Linha do tempo posterior	HPA atual/desejado=5 e deployment=5. A saída não captura explicitamente cinco pods Ready.
Depois	Evaluation retornou a 1/1; CPU 1%. Analytics continuou com uma réplica.

O campo resumido maxEvaluation=2 ficou incompleto porque não acompanhou o pico posterior. O resultado é descrito pela linha do tempo preservada, sem modificar a saída bruta. O vídeo exibe esse resumo e cita o máximo de duas réplicas; a sequência completa está na evidência 13.

O enunciado pede carga com hey, ab ou Postman. O ensaio de evaluation utiliza pods geradores; essa diferença de procedimento fica explícita para conferência da demonstração final.

Ensaio de analytics — 02/11/2026

Fonte: evidência 14, quadros da gravação iniciada às 10:41 BRT. RunId do lote: video-hpa-20261102134256-40a000. O lote foi enviado diretamente à tgmqueue por apresentacao/scripts/04-carga-analytics-sqs.py. São eventos sintéticos, distintos do tráfego HTTP de avaliação.

Etapa	Registro observado
Antes — 10:42	Um pod Running; HPA cpu 2%/30%, réplicas 1.
Envio manual — 10:43	6000 mensagens aceitas pela tgmqueue, em lotes de 10 enviados em paralelo.
Scale-out — 10:43	HPA cpu 340%/30%, réplicas 4. Cinco pods 1/1 Running, quatro deles criados havia 15 a 30 s. kubectl top: 275m a 370m nos quatro pods listados.
Máximo de réplicas — 10:44	HPA cpu 73%/30%, réplicas 5, o máximo configurado.
Persistência — 10:45	Scan do DynamoDB filtrado pelo RunId: 6000 eventos.
Depois — 10:45	HPA cpu 1%/30%, réplicas 1.

Analytics: aumento de réplicas por CPU e retorno ao mínimo comprovados com o envio manual de 6000 mensagens. Em um lote anterior de 100 mensagens, enviado em sequência, o HPA permaneceu em uma réplica: o volume de mensagens, isoladamente, não garante escala.

7. Testes e resultados

Fluxo funcional e rastreabilidade

Etapa	Resultado registrado / limite
Autenticação	Criação e validação de chave no fluxo de testes; valores não incluídos nos documentos.
Flag e targeting	Capturas apresentadas na validação mostram HTTP 201 para a flag load-postman-fiap e regra PERCENTAGE=100.
Avaliação	Captura de Postman mostra HTTP 200, result=true e testes 3/3 para load-postman-fiap.
Persistência AWS	Evidência 07: scan encontrou quatro itens de load-postman-fiap entre 13 itens examinados.
Persistência do lote SQS	Evidência 14: scan filtrado pelo RunId encontrou 6000 eventos em 02/11/2026.
Registro do smoke AWS	EVIDENCIAS_AWS_LAB_2026-10-30.md descreve passagem por SQS/analytics até DynamoDB, sem anexar a saída bruta do identificador desse smoke.
Validação local	Teste local de 31/10 concluiu saúde, autenticação, flag, regra, avaliação e aumento da contagem no DynamoDB local.

As capturas de Postman foram apresentadas durante a validação com a equipe e não integram os TXT anexados. Esses TXT comprovam o inventário, o scan DynamoDB e os ensaios HPA. Não se atribui ao smoke de 31/10 uma contagem DynamoDB que não esteja registrada em saída própria.

Como interpretar os resultados

A resposta da API e a persistência são verificações diferentes. Para um novo teste, utilizar flag_name/user_id únicos, registrar a resposta e localizar o evento correspondente no DynamoDB. Um aumento na contagem total pode ter origem em tráfego concorrente.

Na evidência 07, DescribeTable apresenta ItemCount=0, enquanto o scan encontrou registros. Essa estatística de tabela não deve ser usada para contradizer o resultado da consulta nem para afirmar que a tabela estava vazia.

Entrada observada no LAB 2

http://a5bc56a01f92c4e85887af601f04b84e-1447597342.us-east-1.elb.amazonaws.com

O endereço corresponde à coleta datada; pode mudar se o Load Balancer for recriado. As rotas disponíveis são /auth, /flags, /rules e /evaluate. A raiz / não possui rota de aplicação.

scripts/test-aws-deployment.ps1 automatiza um novo teste e consulta os itens da flag criada. apresentacao/scripts/03-enviar-eventos-sqs.ps1 identifica mensagens por RunId e verifica falhas de envio. 04-carga-analytics-sqs.py faz o mesmo em paralelo, para lotes maiores, e 05-acompanhar-analytics.ps1 registra fila, CPU e réplicas. A existência dos scripts não substitui a saída de sua execução.

8. Evidências

Coletas datadas e arquivos anexos

Arquivo TXT no pacote evidencias/	Conteúdo
01_identidade	Conta e região da sessão AWS.
02_eks_nodegroup	Cluster, node group, escala e nós.
03_ecr	Repositórios e tags das imagens.
04_rds	Três bancos privados e seus estados.
05_elasticache	Redis, TLS, AUTH e cluster mode.
06_sqs	Fila tgmqueue e atributos no instante da coleta.
07_dynamodb	Tabela e scan filtrado por load-postman-fiap.
08_security_groups	Origens e portas autorizadas.
09_kubernetes	Deployments, pods, Ingresses e HPAs.
10_external_secrets	Store e estado dos ExternalSecrets.
11_load_balancer	Classic Load Balancer e duas instâncias InService.
12_secrets_names	Nomes dos segredos, sem seus valores.
13_hpa_antes_durante_depois	Ensaio de evaluation de 31/10, CPU/réplicas e retorno ao mínimo.
14_hpa_analytics_sqs	Ensaio de analytics de 02/11: envio manual, scale-out, retorno ao mínimo e scan pelo RunId. Acompanha os quadros 14a a 14f.

Cronologia

Arquivos 01–12: 30/10/2026, 22:04:41–22:05:20 BRT. Arquivo 13 incluído neste pacote: 31/10/2026, 00:59:22–01:02:31 BRT. O nome da pasta de origem contém 30/10, mas o cabeçalho interno do arquivo 13 identifica corretamente o ensaio de 31/10.

Arquivo 14: 02/11/2026, 10:42–10:45 BRT. Transcreve os valores exibidos no terminal durante a gravação e foi conferido nos quadros 14a a 14f; não é uma saída bruta salva pelos comandos.

Há uma captura anterior do HPA em apresentacao/evidencias, de 30/10, 22:32:20–22:34:49. Ela mostra duas réplicas Ready, uma linha do tempo que chega a três e retorno a uma. É um ensaio diferente e não foi misturado à coleta de 31/10.

Os arquivos 01 a 13 são saídas de AWS CLI e kubectl. As imagens renderizadas dessas saídas, quando usadas na apresentação, não devem ser identificadas como screenshots do Console AWS. No vídeo, as evidências 04 e 08 aparecem identificadas como AWS CLI de 30/10/2026.

Na evidência 08, o IP administrativo foi mascarado para publicação, no arquivo e no vídeo. Nenhuma outra saída foi alterada.

A revisão não substituiu nomes dentro de snapshots históricos. O documento anterior do LAB 1 foi preservado em docs/historico; ele não faz parte do pacote final do LAB 2.

9. Atendimento ao enunciado

Síntese do que está implementado e comprovado

Requisito	Situação documentada
Cinco Dockerfiles	Implementados, multi-stage e execução sem root.
Nove contêineres locais	Validação local positiva. LocalStack é uma adaptação ao DynamoDB Local literal.
EKS / node group / ECR	Inventário datado do LAB 2 anexado.
Três RDS / Redis / SQS / DynamoDB	Recursos e configurações registrados nas evidências.
Metrics Server / Ingress	Métricas e roteamento registrados; Classic difere de ALB/NLB citado no enunciado.
Manifests e boas práticas	Namespaces, Deployments, Services, configurações, segredos, probes e recursos definidos.
HPA evaluation	Aumento de réplicas e retorno ao mínimo registrados no ensaio de 31/10.
HPA analytics	Configurado por CPU; aumento de 1 para 5 réplicas e retorno ao mínimo comprovados em 02/11 (evidência 14 e vídeo, 13:28–15:19).
Envio manual de SQS	Executado na gravação de 02/11: 6000 mensagens aceitas, identificadas pelo RunId video-hpa-20261102134256-40a000.
Persistência DynamoDB	Scan com quatro itens de load-postman-fiap em 30/10 e 6000 eventos do RunId em 02/11.
Arquitetura e decisões (p. 9)	Seção 1: propósitos de RDS, ElastiCache e DynamoDB. Seção 6: implementação do HPA analytics e justificativa da escolha frente ao KEDA.
Vídeo até 20 minutos	Arquivo editado com 18:19, conferido em 02/11; link a preencher após a publicação.
Identificação dos integrantes	Cinco nomes, quatro RMs e cinco usernames do Discord informados; falta o RM de Cesar.

Limites da conclusão

O relatório consolida a implementação e a evidência disponível. Não declara atendimento integral nas diferenças de produto e procedimento explicitadas acima: Classic Load Balancer frente a ALB/NLB, LocalStack frente a DynamoDB Local e pods geradores frente a hey, ab ou Postman.

No vídeo, a criptografia do RDS e as regras das portas 5432 e 6379 aparecem pelas evidências 04 e 08, de 30/10. O deployment do Metrics Server e a release Helm do ingress-nginx não são exibidos; a disponibilidade de métricas aparece nos valores de CPU do HPA e no kubectl top.

O badge público do Google Cloud Skills Boost é opcional para pontuação adicional. Nenhuma conclusão de trilha ou URL de badge foi presumida.

10. Reprodução e consulta

Comandos compatíveis com Windows PowerShell 5.1

Executar um bloco por vez na raiz do repositório. O pacote de documentos é material de entrega; a execução depende do projeto completo, Docker Desktop e, para AWS, sessão de laboratório válida.

Selecionar a pasta de trabalho

Neste computador, usar a cópia existente:

```
Set-Location "C:\Users\Jeferson\Desktop\Fase 2 Tech FIAP"
```

Em outro computador, obter uma nova cópia e entrar na pasta fase2. A Fase 1 permanece na raiz do repositório:

```
git clone https://github.com/jefersonpapalinosp-sys/Tech-challenge-grupo-2.git
Set-Location .\Tech-challenge-grupo-2\fase2
```

Ambiente local — executar na cópia escolhida

```
docker compose up -d --wait --wait-timeout 240
docker compose ps
(docker compose ps -q | Measure-Object).Count
foreach ($porta in 18001,18002,18003,18004,18005) {
    Write-Host "Porta $porta"
    Invoke-RestMethod "http://localhost:$porta/health"
}
```

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\scripts\test-local.ps1
```

O teste cria registros locais de demonstração. Não é necessário apagar volumes para iniciar ou apresentar o ambiente. docker compose down -v remove os dados dos volumes.

Consultar a implantação AWS

No ambiente de Jeferson, a preparação abaixo usa as credenciais do LAB guardadas fora do pacote de entrega. A liberação de scripts aplica-se somente ao processo atual.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
. .\apresentacao\scripts\01-preparar-sessao.ps1
kubectl config current-context
kubectl get nodes
kubectl get deployments -A -l app.kubernetes.io/part-of=togglemaster
kubectl get externalsecret -A
kubectl get hpa -A
kubectl get svc my-release-ingress-nginx-controller -n ingress-nginx
```

O arquivo apresentacao/08_COMANDOS_SOMENTE_PARTE_B.txt reúne a sequência da apresentação. Os trechos antigos com ForEach-Object -Parallel exigiam PowerShell 7 e foram retirados desta versão.

11. Entrega e referências

Arquivos finais e campos reservados

A pasta apresentacao/entrega_final contém o relatório em Word, PDF e TXT, as evidências TXT, os quadros do ensaio de 02/11 e instruções curtas de preenchimento. O Word é a versão editável. PDF ou TXT podem compor a entrega formal prevista no enunciado; o PDF deve ser exportado novamente se o Word for alterado.

Preencher antes de enviar

Campo	Situação
RM e Discord	Falta o RM de Cesar Augusto Ferreira Alves; o Discord dos cinco está preenchido.
Link do vídeo	A inserir após publicação; validar acesso pelo avaliador.
Duração final	18:19 no arquivo conferido em 02/11; atualizar se o vídeo for editado de novo.
Conteúdo demonstrado	Conferido em 02/11: envio manual SQS, escala do analytics e eventos do lote no DynamoDB.
Scripts 04 e 05	Publicados em fase2/apresentacao/scripts, com os comandos do ensaio em COMANDOS_ANALYTICS_HPA.txt.
Link do repositório	Destino do grupo definido como Tech-challenge-grupo-2; conferir acesso e conteúdo antes da submissão.

Referências do projeto

Repositório do grupo: https://github.com/jefersonpapalinosp-sys/Tech-challenge-grupo-2

Código-fonte da Fase 2: https://github.com/jefersonpapalinosp-sys/Tech-challenge-grupo-2/tree/main/fase2

Histórico de publicações da Fase 2: https://github.com/jefersonpapalinosp-sys/Tech-challenge-grupo-2/commits/main/fase2

docs/POSTECH - Tech Challenge - Fase 2.pdf — requisitos técnicos e entregáveis.

docker-compose.yml e os cinco diretórios *-service/Dockerfile — construção e execução local.

k8s-aws/ — implantação, configurações, Ingresses, ExternalSecrets e HPAs.

docs/evidencias_aws_2026-10-30/ — fontes brutas do inventário e do ensaio de HPA.

docs/evidencias_aws_2026-11-02/ — ensaio de escala do analytics.

docs/EVIDENCIAS_AWS_LAB_2026-10-30.md — registro narrativo do fluxo funcional no LAB 2.

apresentacao/ — roteiro, comandos e material de apoio da gravação.

Controle da revisão

Esta revisão substitui o conteúdo operacional do relatório de 29/10 pelo registro do LAB 2. A versão anterior foi preservada como histórico. As informações de conta, recursos, namespaces, tipo de Load Balancer e escalabilidade foram reconciliadas com as fontes atuais do projeto.

A revisão de 02/11/2026 acrescentou o ensaio de analytics e a conferência do vídeo editado, e publicou o relatório, a conferência, as evidências e os scripts 04 e 05 em fase2. A versão de 31/10 foi preservada na cópia local, em docs/historico.

A documentação não contém valores de credenciais. A redação do relatório não executou provisionamento, exclusão, carga ou mudanças de configuração na AWS. A carga de 02/11 foi executada na gravação, e o ajuste dos nós está descrito na seção 4.